

VPC and Networking

Subnets · Security Groups · Route Tables · NAT Gateway · VPC Endpoints · Flow Logs

AWS Tutorial · ShopFlow

03-00

Chapter Overview — VPC and Networking

A VPC (Virtual Private Cloud) is your own private section of the AWS cloud — like having a walled data center inside AWS. Everything runs inside your VPC: web servers, application containers, databases, caches. You control: which resources are exposed to the internet, which can only talk to each other privately, and what traffic is allowed on which ports. ShopFlow lives entirely inside a VPC; customers reach it only through the load balancer, never directly.

■ VPC in Plain English

Imagine AWS is a city with thousands of buildings (servers). Without a VPC, your servers are on a public street — anyone can walk up. A VPC is like building a private compound with walls, gates, and security guards. You decide which gates (load balancers) face the street, which buildings (databases) are hidden inside, and who is allowed through each gate.

Example: ShopFlow VPC: 10.0.0.0/16. The load balancer sits at the public gate. ECS containers are in a private building. The Aurora database is in a locked vault with no external door. Only the containers can reach the database, and only on port 5432.

VPC Component	What It Is	ShopFlow Example
VPC	Your private network in AWS with an IP address range (CIDR block) up to 65,536 IP addresses in the ShopFlow network.	10.0.0.0/16
Subnet	A subdivision of the VPC in one Availability Zone	10.0.1.0/24 (public subnet in us-east-1a) — 256 IPs for load balancer
Internet Gateway	Allows resources in public subnets to reach the internet	Attached to ShopFlow VPC — enables ALB to receive customer requests
NAT Gateway	Allows private subnet resources to reach the internet (outbound only)	ECS tasks can load npm packages and call external APIs via NAT
Route Table	Rules that decide where network traffic is sent	Public subnet: 0.0.0.0/0 → Internet Gateway. Private subnet: 0.0.0.0/0 → NAT Gateway
Security Group	Stateful firewall rules for individual resources (EC2, RDS, ECS)	allow-alb-sg: port 443 from 0.0.0.0/0. allow-ecs-sg: port 8080 from 10.0.0.0/16
NACL	Stateless firewall rules for entire subnets	Block known malicious IPs at the subnet level before they reach resources

03-01

ShopFlow VPC Design — 3-Tier Architecture

ShopFlow uses a classic 3-tier VPC: public tier (internet-facing load balancer), application tier (ECS containers — no direct internet access), and data tier (Aurora, DynamoDB, ElastiCache — completely private). This layout ensures that even if an attacker breaks into a web server, they cannot directly reach the database.

Tier	Subnet Type	Resources	CIDR Blocks
Public (Tier 1)	Public subnets — have internet access	Application Load Balancer, NAT Gateways	10.0.1.0/24, 10.0.2.0/24, 10.0.3.0/24 (one per AZ)
Application (Tier 2)	Private subnets — outbound only via NAT	ECS tasks (all 8 microservices)	10.0.11.0/24, 10.0.12.0/24, 10.0.13.0/24 (one per AZ)
Data (Tier 3)	Isolated private subnets — no internet access	Aurora PostgreSQL, ElastiCache Redis, OpenSearch	10.0.21.0/24, 10.0.22.0/24, 10.0.23.0/24 (one per AZ)

ShopFlow VPC — 10.0.0.0/16 (us-east-1)

Internet Gateway — entry point for all customer traffic

Public Subnets — 10.0.1-3.0/24 (one per AZ)

Application Load Balancer — HTTPS :443, HTTP :80

NAT Gateways — 3x (one per AZ) for private subnet outbound

Private App Subnets — 10.0.11-13.0/24

ECS Fargate: order-svc, product-svc, user-svc, inventory-svc

ECS Fargate: payment-svc, notification-svc, search-svc

Private Data Subnets — 10.0.21-23.0/24 (isolated)

Aurora PostgreSQL cluster (writer + 2 readers)

ElastiCache Redis cluster (3 shards)

OpenSearch domain (3 nodes)

03-02

Creating the ShopFlow VPC in the Console

AWS provides a VPC Wizard that creates a complete VPC with subnets, route tables, and internet gateway in one step. For ShopFlow we will use the wizard and then customize. This takes about 10 minutes in the console.

■ Create the ShopFlow VPC (Console Wizard)

1. Search "VPC" in the AWS console → Click "Create VPC"
2. Resources to create: VPC and more (this creates subnets, route tables, IGW all at once)
3. Name tag auto-generation: shopflow
4. IPv4 CIDR: 10.0.0.0/16
5. Number of Availability Zones: 3
6. Number of public subnets: 3 (will be 10.0.0.0/24, 10.0.1.0/24, 10.0.2.0/24)
7. Number of private subnets: 3 (will be 10.0.128.0/24, 10.0.129.0/24, 10.0.130.0/24)
8. NAT gateways: 1 per AZ (select this for high availability — costs ~\$100/month)
9. VPC endpoints: S3 Gateway (free — saves NAT Gateway data transfer costs)
10. Click Create VPC — takes about 2 minutes to create all resources

Create VPC — VPC settings

Resources to create

■ VPC only ■ VPC and more

Name tag auto-generation

shopflow

■ Prefixes all created resources (shopflow-vpc, shopflow-subnet-public1-us-east-1a, etc.)

IPv4 CIDR block

10.0.0.0/16

■ This gives you 65,536 IP addresses. /16 is standard for a production VPC.

■ Availability Zones and Subnets

Number of Availability Zones (AZs)

3 ■

Number of public subnets

3 ■

Number of private subnets

3 (we will add a 3rd private tier manually) ■

■ NAT gateways and VPC Endpoints

NAT gateways

■ None (free but private subnets have no internet) ■ 1 per VPC ■ 1 per AZ (recommended for production — survives AZ failure)

■ S3 Gateway endpoint — free; routes S3 traffic inside AWS network (faster, no NAT cost)

Create VPC

03-03

Security Groups — The ECS and Database Firewall

Security Groups are virtual firewalls attached to individual resources. They are stateful: if you allow inbound traffic on port 443, the response traffic is automatically allowed out without a separate rule. ShopFlow has distinct security groups for each tier, each allowing only the specific traffic needed.

■ Create Security Groups for ShopFlow

1. VPC → Security groups → Create security group
2. Name: shopflow-alb-sg | VPC: shopflow-vpc
3. Inbound rules: HTTPS (443) from 0.0.0.0/0 + HTTP (80) from 0.0.0.0/0
4. Outbound rules: All traffic (default) → click Create security group
5. Create another: shopflow-ecs-sg | VPC: shopflow-vpc
6. Inbound: Custom TCP port 8080 from Source: shopflow-alb-sg (select the SG, not an IP)
7. Outbound: All traffic (ECS tasks need outbound to call AWS APIs and external services)
8. Create another: shopflow-aurora-sg | VPC: shopflow-vpc
9. Inbound: PostgreSQL (5432) from Source: shopflow-ecs-sg only
10. Outbound: None (databases should not initiate outbound connections)

Security Group	Inbound Rules	Outbound Rules	Attached To
shopflow-alb-sg	HTTPS :443 from 0.0.0.0/0 (any IP) HTTP :80 from 0.0.0.0/0 (redirect to HTTPS)	All traffic (load balancer needs to reach EC2)	Application Load Balancer
shopflow-ecs-sg	TCP :8080 from shopflow-alb-sg only (containers only accept traffic from ALB)	All traffic (containers call AWS APIs, external services, etc)	AlexaRankingService databases
shopflow-aurora-sg	PostgreSQL :5432 from shopflow-ecs-sg only (database only accepts from ECS tasks)	None (database never initiates connections)	Aurora PostgreSQL cluster
shopflow-redis-sg	Redis :6379 from shopflow-ecs-sg only	None	ElastiCache Redis cluster
shopflow-opensearch-sg	HTTPS :443 from shopflow-ecs-sg only	None	OpenSearch domain

03-04

Security Groups in CDK

In CDK, security groups are created with the `ec2.SecurityGroup` construct. You add rules with `addIngressRule` and `addEgressRule`. The most powerful feature: you can reference one security group as the source of another, which is far more precise than specifying IP ranges.

```
import * as ec2 from "aws-cdk-lib/aws-ec2";
// Create the VPC (equivalent to the console wizard above)
const vpc = new ec2.Vpc(this, "ShopFlowVpc", {
  vpcName: "shopflow-vpc",
  cidr: "10.0.0.0/16",
  maxAzs: 3,
  natGateways: 3, // 1 per AZ for HA
  subnetConfiguration: [
    {name:"public", subnetType: ec2.SubnetType.PUBLIC, cidrMask: 24},
    {name:"private", subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS, cidrMask: 24},
    {name:"data", subnetType: ec2.SubnetType.PRIVATE_ISOLATED, cidrMask: 24},
  ],
});
// ALB security group: public-facing HTTPS
const albSg = new ec2.SecurityGroup(this, "AlbSg", {
  vpc, securityGroupName: "shopflow-alb-sg",
  description: "ALB: accept HTTPS from anywhere",
});
albSg.addIngressRule(ec2.Peer.anyIpv4(), ec2.Port.tcp(443), "HTTPS from internet");
albSg.addIngressRule(ec2.Peer.anyIpv4(), ec2.Port.tcp(80), "HTTP redirect");
// ECS security group: accept only from ALB
```

```
const ecsSg = new ec2.SecurityGroup(this, "EcsSg", {
  vpc, securityGroupName: "shopflow-ecs-sg",
});
ecsSg.addIngressRule(albSg, ec2.Port.tcp(8080), "From ALB only");
// Aurora security group: accept only from ECS
const auroraSg = new ec2.SecurityGroup(this, "AuroraSg", {
  vpc, securityGroupName: "shopflow-aurora-sg",
});
auroraSg.addIngressRule(ecsSg, ec2.Port.tcp(5432), "PostgreSQL from ECS only");
```

03-05

Route Tables — Directing Traffic Through the VPC

Route Tables define where network packets go. Each subnet is associated with one route table. Public subnets route internet-bound traffic to the Internet Gateway. Private subnets route it to the NAT Gateway (outbound only). Data subnets have no internet route at all — the most isolated tier.

Subnet Type	Route Table Rules	Effect
Public subnets (10.0.1-3.0/24)	10.0.0.0/16 → local (VPC traffic stays in VPC) 0.0.0.0/0 → igw-xxxxx (Internet Gateway)	Resources here (ALB) can receive traffic from internet
Private app subnets (10.0.11-13.0/24)	10.0.0.0/16 → local 0.0.0.0/0 → nat-xxxxx (NAT Gateway)	ECS tasks can reach internet (for AWS APIs, npm)
Private data subnets (10.0.21-23.0/24)	10.0.0.0/16 → local (no 0.0.0.0/0 route — no internet)	Aurora, Redis, OpenSearch: completely isolated;

■ Verify Route Tables in the Console

1. VPC → Route tables → look for shopflow-rtb-public, shopflow-rtb-private-us-east-1a, etc.
2. Click shopflow-rtb-public → Routes tab: you should see 0.0.0.0/0 → igw-xxxxx
3. Click shopflow-rtb-private → Routes tab: you should see 0.0.0.0/0 → nat-xxxxx
4. Click Subnet associations tab: verify each subnet is associated with the correct route table
5. If a private subnet shows igw as target for 0.0.0.0/0, it is accidentally public — fix immediately

03-06

VPC Endpoints — Connect to AWS Without Internet

VPC Endpoints let resources in private subnets call AWS services (S3, DynamoDB, Secrets Manager, ECR, CloudWatch) without going through the internet (NAT Gateway). This is faster, cheaper, and more secure. ShopFlow saves approximately \$150/month in NAT Gateway data transfer costs by using VPC endpoints for high-traffic AWS API calls.

VPC Endpoint	Type	What It Enables	Monthly Saving
S3 Gateway	Gateway (free)	ECS tasks access S3 product images without NAT Gateway	\$45/mo (\$1045/ShopFlow scale)
DynamoDB Gateway	Gateway (free)	ECS tasks access DynamoDB without NAT Gateway	~\$30/month
ECR API + DKR	Interface (~\$16/month)	ECS pulls container images from ECR without NAT Gateway	\$60/mo (\$609/mo saved)
Secrets Manager	Interface (~\$16/month)	ECS reads secrets without internet; better security posture	Security benefit + ~\$5/month
CloudWatch Logs	Interface (~\$16/month)	ECS writes logs without internet; data isolated in AWS	Security benefit + ~\$10/month

```
// Add VPC endpoints in CDK
// Gateway endpoints (free — just a route table entry)
vpc.addGatewayEndpoint("S3Endpoint", {
  service: ec2.GatewayVpcEndpointAwsService.S3,
});
```

```

vpc.addGatewayEndpoint("DynamoEndpoint", {
  service: ec2.GatewayVpcEndpointAwsService.DYNAMODB,
});
// Interface endpoints (hourly cost per AZ – use only for high-traffic APIs)
vpc.addInterfaceEndpoint("SecretsManagerEndpoint", {
  service: ec2.InterfaceVpcEndpointAwsService.SECRETS_MANAGER,
  subnets: {subnetType: ec2.SubnetType.PRIVATE_WITH_EGRESS},
});

```

03-07

Network ACLs vs Security Groups

NACLs (Network Access Control Lists) and Security Groups are both firewalls, but at different levels. Security Groups are stateful (response traffic auto-allowed) and attached to individual resources. NACLs are stateless (you must explicitly allow response traffic) and applied to entire subnets. Use Security Groups for normal access control; use NACLs to block entire IP ranges (DDoS mitigation).

	Security Groups	Network ACLs
Applied to	Individual resources (ECS task, RDS instance, ALB, etc.)	Entire subnets (everything in the subnet)
State	Stateful: response traffic is auto-allowed; you only define inbound rules	Stateless: you must explicitly allow both inbound AND outbound for every rule
Rule processing	All rules evaluated; most permissive wins	Rules evaluated in order (lowest number first); first match wins — or deny
Default behavior	No inbound traffic allowed by default; all outbound allowed	All traffic allowed inbound and outbound (AWS default NACL)
Use case	Day-to-day access control (which SG can talk to which)	Block known malicious IPs at subnet level; coarse-grained subnet level
ShopFlow usage	All access control done via Security Groups (primary)	NACL on public subnets: block geographic IPs during DDoS attack

03-08

VPC Flow Logs — See All Network Traffic

VPC Flow Logs capture metadata about every network connection in your VPC: source IP, destination IP, port, protocol, bytes transferred, and whether it was accepted or rejected. Essential for: security investigation (which IP connected to the database?), troubleshooting connectivity issues (why is ECS rejecting traffic?), and cost analysis (how much data is crossing AZs?).

■ Enable VPC Flow Logs for ShopFlow

1. VPC → Your VPCs → select shopflow-vpc → Actions → Create flow log
2. Filter: All (capture accepted and rejected traffic)
3. Maximum aggregation interval: 1 minute (10 min is default; 1 min is better for debugging)
4. Destination: Send to CloudWatch Logs
5. Log group: /aws/vpc/flowlogs/shopflow (create new)
6. IAM role: Create new role (AWS creates a role allowing VPC to write to CloudWatch)
7. Log record format: AWS default format (or custom to add traffic-path and pkt-srcaddr)
8. Click Create flow log — logs appear in CloudWatch within 5-10 minutes

```

# Useful CloudWatch Logs Insights query for VPC Flow Logs
# "Which IPs had the most rejected connections to the database?"
fields srcAddr, dstAddr, dstPort, action, bytes
| filter dstPort = 5432 and action = "REJECT"
| stats count() as rejections by srcAddr
| sort rejections desc
| limit 20

```

■ Enable VPC Flow Logs from day one, even in dev. The first time you have a connectivity issue (ECS task cannot connect to RDS), flow logs will tell you exactly what is being rejected and by which security group rule, saving hours of debugging.

03-09

AWS PrivateLink — Expose Services Without Public Internet

PrivateLink lets you expose a service from your VPC to consumers (other VPCs, partner companies, AWS services) without those connections ever touching the public internet. ShopFlow uses PrivateLink to expose the payment-svc API to a partner financial services company — the partner accesses ShopFlow payment APIs via a private IP in their own VPC, never crossing the internet.

■ PrivateLink vs VPC Peering

VPC Peering connects two VPCs so all resources can talk to each other (like merging two networks). PrivateLink exposes only a specific service (via a load balancer) to specific consumers. PrivateLink is preferred for partner integrations because it does not expose your entire VPC, only the specific endpoint you choose.

Example: ShopFlow payment-svc exposed via PrivateLink: partner company creates a VPC Endpoint in their VPC pointing to the ShopFlow PrivateLink service. Their payment processor calls `https://payment.vpce-xxx.shopflow.vpce.us-east-1.vpce.amazonaws.com` — traffic stays entirely within AWS backbone, never on internet.

Use Case	VPC Peering	PrivateLink
Connect two internal AWS accounts (Dev and Prod)	Yes (VPC Peering); full connectivity between accounts	Overkill; PrivateLink is for exposing specific services, not general connectivity
Expose payment API to a payment partner	No (VPC Peering exposes entire VPC (Security Risk))	Yes (PrivateLink exposes only the NLB endpoint for payment-svc)
Multi-tenant SaaS (serve multiple customers)	No (VPC Peering scale to hundreds of peerings is complex)	Yes (PrivateLink scales to thousands of consumers from a single endpoint)
Cross-region connectivity	Possible but complex (inter-region peering)	Not supported cross-region (use VPN or Direct Connect for cross-region)

03-10

Transit Gateway — Connect Many VPCs

As ShopFlow grows to multiple VPCs (one per environment, plus shared services), connecting them with VPC Peering becomes complex — $N \times (N-1) / 2$ peering connections needed. Transit Gateway is a hub: every VPC connects to the TGW, and the TGW routes traffic between them. ShopFlow eventually connects 6 VPCs (Dev, Staging, Prod, Security, Tools, DR) through one Transit Gateway.

Scenario	Without Transit Gateway	With Transit Gateway
6 VPCs need to communicate (Dev, Staging, Prod, Security, Tools, DR)	15 VPC peering connections needed; manual TGW attachments become cumbersome	One TGW attachment per VPC: 10.0.0.0/8 → TGW
New VPC added (e.g., Data Science)	Must manually peer to every other VPC (5 new connections)	One TGW attachment; add one route; done in 5 minutes
Inspect traffic between VPCs (security)	Complex: must insert inspection VPC at each peering path	Routing path through an inspection VPC at TGW
Monthly cost	\$0 (VPC Peering is free)	~\$35/month per attachment (6 VPCs = ~\$210/month) — worth it

03-11

Hands-On Lab — Build ShopFlow Network from Scratch

This lab walks through building the entire ShopFlow VPC manually in the console so you understand each component before letting CDK manage it. Estimated time: 30 minutes.

Step	What To Do	Console Location
1	Create VPC: CIDR 10.0.0.0/16, name shopflow-vpc	VPC → Your VPCs → Create VPC
2	Create Internet Gateway: name shopflow-igw → Attach to shopflow-vpc	VPC → Internet Gateways → Create IGW
3	Create 3 public subnets: 10.0.1-3.0/24 in us-east-1a/b/c	VPC → Subnets → Create subnet
4	Create 3 private app subnets: 10.0.11-13.0/24 in us-east-1a/b/c	VPC → Subnets → Create subnet

Step	What To Do	Console Location
5	Create 3 private data subnets: 10.0.21-23.0/24 in us-east-1a/b/c	VPC → Subnets → Create subnet
6	Create Elastic IPs for NAT Gateways: 3x (one per AZ)	EC2 → Elastic IPs → Allocate
7	Create 3 NAT Gateways in each public subnet, assign Elastic IP	VPC → NAT Gateways → Create NAT Gateway
8	Public route table: create, add route 0.0.0.0/0 → IGW, associate with public subnets	VPC → Route Tables → Create → add route → Subnet Associations
9	Private route tables (3x): create one per AZ, add 0.0.0.0/0 → NAT GW in that AZ (repeat 3x with correct NAT GW per AZ)	VPC → Route Tables → Create
10	Create 5 security groups (alb-sg, ecs-sg, aurora-sg, redis-sg, opensearch-sg)	VPC → Security Groups → Create (5x)
Verify	VPC → Your VPCs → shopflow-vpc → view Resource map tab	Resource map shows VPC topology visually

03-12

Cross-AZ Data Transfer Costs and Optimization

AWS charges \$0.01/GB for data transferred between Availability Zones within the same region. At ShopFlow scale (millions of requests daily), cross-AZ traffic can add up to hundreds of dollars per month. Design your architecture to minimize unnecessary cross-AZ hops.

Cross-AZ Traffic Source	Monthly Cost Estimate	Optimization
ECS task in AZ-A calling ElastiCache in AZ-B (per cache miss)	\$20.50/month	Deploy ElastiCache with 3 shards (one per AZ); configure client to prefer local shard
ALB distributing traffic evenly to all AZs regardless of origin	\$10.80/month	Enable ALB cross-zone load balancing (default ON); acceptable cost for scale
ECS task in AZ-A calling Aurora writer in AZ-B (only writes go to writer)	\$60.40/month	Can't avoid for writes; for reads, use Aurora reader endpoint which routes to local AZ
NAT Gateway data processing (\$0.045/GB) for internet-bound traffic	\$50-150/month	Use VPC Gateway Endpoints for S3 and DynamoDB (free); use interface endpoints for others

AZ Affinity Pattern

Configure all ShopFlow services to prefer resources in the same Availability Zone. ECS placement with the "spread" strategy puts tasks evenly across AZs. Clients (Redis, Aurora) configured with AZ-aware routing pick the replica in their own AZ first. This reduces cross-AZ traffic by 60-70% at ShopFlow scale.

Example: order-svc in us-east-1a → reads from Redis replica in us-east-1a (same AZ, free) → writes go to Aurora writer in us-east-1b (cross-AZ, unavoidable for writes) → order-svc reads from Aurora reader in us-east-1a (same AZ, free).

03-13

Direct Connect and VPN — Hybrid Connectivity

ShopFlow headquarters has an on-premises data center with legacy inventory systems that need to talk to the ShopFlow VPC. Two options: AWS Site-to-Site VPN (encrypted tunnel over internet, quick to set up, ~\$50/month) or AWS Direct Connect (dedicated fiber connection, lower latency, higher cost starting at ~\$400/month). ShopFlow starts with VPN and plans Direct Connect after launch.

■ Set Up Site-to-Site VPN for ShopFlow

1. VPC → Virtual Private Gateways → Create virtual private gateway → attach to shopflow-vpc
2. VPC → Customer Gateways → Create customer gateway → IP: (your on-prem router public IP) → BGP ASN: 65000
3. VPC → Site-to-Site VPN connections → Create VPN connection
4. Target: Virtual Private Gateway (the one you created)
5. Customer gateway: the one you created
6. Routing: Static → add routes for your on-prem network (e.g., 192.168.0.0/16)
7. Download the configuration file for your on-prem router (Cisco, Juniper, Palo Alto, etc.)
8. Apply the configuration to your on-prem router → VPN tunnels go UP (green)
9. Add route in private subnets route table: 192.168.0.0/16 → Virtual Private Gateway
10. Verify: from on-prem, ping an ECS task private IP (e.g., 10.0.11.5)

	Site-to-Site VPN	AWS Direct Connect
Setup time	30 minutes	30-90 days (physical fiber provisioning)
Bandwidth	Up to 1.25 Gbps (limited by internet)	1 Gbps to 100 Gbps (dedicated fiber)
Latency	Variable (shared internet)	Consistent, low latency (~5ms vs ~25ms for internet)
Reliability	99.95% SLA (dual tunnels per connection)	99.99% SLA with redundant connections
Cost	~\$50/month + \$0.05/GB data transfer	~\$400-2000/month (port fee) + data transfer
ShopFlow usage	Connects ShopFlow VPC to on-prem legacy IP	Planned for 12 months migration period which data volume justifies cost

03-14

Network Monitoring with VPC Reachability Analyzer

Before deploying a new ECS service, use VPC Reachability Analyzer to verify that your security group and route table configuration actually allows the intended traffic. It analyzes network paths without sending real traffic — no packets are created, no deployments needed. This prevents the frustrating scenario of deploying order-svc and discovering it cannot reach the Aurora database.

■ Verify ECS-to-Aurora Connectivity with Reachability Analyzer

1. VPC → Network Manager → Reachability Analyzer → Create and analyze path
2. Source type: Network Interface → select an ECS task ENI (or the ECS security group)
3. Destination type: Network Interface → select an Aurora instance ENI
4. Protocol: TCP | Destination port: 5432
5. Click Create and analyze path
6. Result: Reachable (green) or Not reachable (red with explanation)
7. If not reachable: the analyzer shows exactly which hop blocked it (missing route, SG deny, NACL deny)
8. Fix the issue (add SG rule, fix route table) → re-analyze → verify Reachable

Reachability Analyzer Finding	Root Cause	Fix
Blocked by security group inbound rule	Aurora SG does not have port 5432 allowed from ECS SG	Add EC2 SG rule: Custom TCP :5432 from shopflow

Reachability Analyzer Finding	Root Cause	Fix
No route in route table	Data subnet route table has no local route or is missing	Verify route table has 10.0.0.0/16 → local; check sub
Blocked by Network ACL	NACL on data subnet explicitly denies traffic	Check NACL inbound rules; NACL rules are evaluat
Instance not in expected subnet	Aurora cluster placed in wrong subnet (public instead of data)	Recreate Aurora in the correct private data subnet

03-15 ShopFlow Network Architecture — Complete Picture

Here is the complete network flow for a customer placing an order on ShopFlow. Every hop goes through a security check. No component has more network access than it needs.

Step	Network Path	Security Check
1. Customer HTTP request	Internet Gateway → ALB (public subnet)	WAF: checks for SQL injection, XSS, known bad IPs
2. ALB forwards to ECS	shopflow-alb-sg → ECS task (private app subnet)	SG: only shopflow-alb-sg allowed on port 8080; all other sources rejected
3. order ECS task → DB	ECS task → RDS/DynamoDB VPC Gateway Endpoint (no internet)	IAM policy: only GetItem/PutItem on shopflow-orders table
4. order ECS task → Secrets Manager	ECS task → Secrets Manager VPC Interface Endpoint	IAM: only GetSecretValue on specific secret ARN
5. order ECS task → Aurora	ECS task → Aurora (data subnet 10.0.21.x)	SG: shopflow-aurora-sg allows only shopflow-ecs-sg on port 5432
6. order ECS task → SQS	ECS task → SQS (SQS managed, no VPC required)	IAM: only sqs:SendMessage on shopflow-order-queue ARN
7. Content logs to CloudWatch	ECS task → CloudWatch Logs Interface Endpoint	IAM: logs:PutLogEvents on /shopflow/order-svc log group only
8. Response to customer	Aurora → ECS → ALB → Internet Gateway → customer	TLS encryption for entire path; end-to-end encrypted

03-16 IPv6 Support in ShopFlow VPC

AWS VPCs support dual-stack networking (IPv4 and IPv6). While ShopFlow primarily uses IPv4 internally, IPv6 is important for future-proofing and for reaching services that are moving to IPv6-only. ECS Fargate supports IPv6, and enabling it is straightforward. The key difference: IPv6 subnets are always publicly routable (there is no IPv6 NAT), so you must use Security Groups carefully.

IPv6 Consideration	IPv4 Behavior	IPv6 Behavior
Address assignment	Private IPs (10.x.x.x) for internal; Elastic IP for external	IPv6 addresses are globally unique and publicly routable; no disti
Internet routing	Public subnet: IGW routes public IPs. Private subnet: NAT Gateway (S	Egress Only: NAT Gateway (S
Security isolation	Private subnets protected by no-internet	Isolation achieved via Security Groups (no inbound rules) + EIG
ECS Fargate	Default networking mode uses IPv4	Enable dualStackIPv6 in VPC; set assignPublicIp to DISABLED; Sec

```
// Enable IPv6 in CDK VPC
const vpc = new ec2.Vpc(this, "ShopFlowVpc", {
  cidr: "10.0.0.0/16",
  maxAzs: 3,
  // Enable IPv6 dual-stack
  enableDnsHostnames: true,
  enableDnsSupport: true,
});
// Add IPv6 CIDR block to the VPC (AWS assigns /56 automatically)
const ipv6Cidr = new ec2.CfnVPCIPv6CidrBlock(this, "IPv6Cidr", {
  vpcId: vpc.vpcId,
  amazonProvidedIpv6CidrBlock: true,
});
```

03-17 Troubleshooting Common VPC Connectivity Issues

Connectivity issues are the most common problem when building on AWS. This section covers the systematic approach to diagnosing and fixing "ECS task cannot reach database" and "load balancer returning 502" errors — the two most frequent issues in ShopFlow deployments.

Symptom	Likely Cause	Diagnostic Steps	Fix
ECS task: connection refused Aurora 5432	Wrong SG inbound rule for ECS SG	1. Check ECS SG inbound rules 2. Check ECS task is in expected SG 3. Use Reachability Analyzer	Add inbound rule: TCP 5432 from shopflow
ALB returning 502 Bad Gateway	ECS task not accepting connections on 8080	1. Check ECS 8080 is running (ECS console) 2. Check health check path returns HTTP 200 3. Check ECS SG allows port 8080 from ALB SG	Fix app to listen on 0.0.0.0:8080; check he
ECS task: cannot pull image from ECR	Private subnet has no internet; ECR role missing	1. Create NAT GW in public 2. Check route table has 0.0.0.0/0 → NAT 3. Check ECR endpoint exists	Create ECR API and DKR interface VPC e
ECS task: cannot write logs to CloudWatch	CloudWatch logs endpoint missing	1. Check ECS task role has logs SuperGrossLog 2. Check CloudWatch Logs VPC endpoint exists	PutLogEvents on task role; add
RDS: connection timeout (not refused)	Refused if missing or NACL blocking	1. Check logs show REJECT at subnet level 2. Check NACL rules 3. Check route table	Fix NACL rule ordering; verify private subnet

03-18

Multi-Region VPC Architecture — Disaster Recovery

ShopFlow active-active architecture spans us-east-1 (primary) and us-west-2 (secondary). Each region has an identical VPC layout. Cross-region connectivity uses VPC Peering (low volume control-plane traffic) or AWS Global Accelerator (customer-facing routing). The ShopFlow DR VPC uses 10.1.0.0/16 (not 10.0.0.0/16) to avoid CIDR overlap for future peering.

Architecture Component	us-east-1 (Primary)	us-west-2 (DR)
VPC CIDR	10.0.0.0/16	10.1.0.0/16 (different range — required for peering)
Public subnets	10.0.1-3.0/24	10.1.1-3.0/24
Private app subnets	10.0.11-13.0/24	10.1.11-13.0/24
Private data subnets	10.0.21-23.0/24	10.1.21-23.0/24
Aurora	Writer + 2 readers (active)	Global Database secondary (read-only unless failover)
ECS tasks	All 8 services running (full capacity)	All 8 services running (standby; scaled down to 1 task ea
Traffic	95% of customer traffic via Route 53 → us-east-1	5% of health checks; auto-promoted to 100% if us-east-1

03-19

Network Cost Analysis for ShopFlow

Networking is often an overlooked cost on AWS. ShopFlow network costs at 1M MAU with 1,000 req/s average traffic — understanding where money goes helps you optimize.

Network Cost Item	Calculation	Monthly Cost
NAT Gateway hours (3x AZ)	\$0.045/hr × 3 × 730 hrs	\$98/month
NAT Gateway data processing	\$0.045/GB × ~500 GB (before endpoint optimization)	\$22/month (reduced to ~\$8 with VPC endpoint
VPC Interface Endpoints (ECR, Secrets, CloudFront)	\$0.015/hr × 3 AZs × 3 endpoints × 730 hrs	\$66/month
Cross-AZ data transfer	\$0.01/GB × ~200 GB (ECS to Aurora cross-AZ read)	\$2/month
Internet data transfer (egress to customers)	\$0.09/GB (first 10TB) × ~2 TB product images via ALB	\$180/month (switch to CloudFront to reduce l

Network Cost Item	Calculation	Monthly Cost
VPC Flow Logs (CloudWatch ingestion)	\$0.50/GB × ~10 GB/month logs	\$5/month
Total estimated network cost		~\$370/month (before CloudFront optimization)

■ Network Cost Quick Win

Moving product image traffic from ALB direct to CloudFront reduces egress costs from \$180/month to ~\$36/month (CloudFront pricing is \$0.0085/GB vs \$0.09/GB direct). CloudFront also caches images globally, reducing origin load. This single change saves ~\$144/month and improves customer experience. See Chapter 5 (S3 and CloudFront) for implementation.

03-20 VPC CIDR Planning for Growing Teams

Poor CIDR planning is one of the most expensive mistakes in AWS architecture. You cannot change a VPC CIDR after creation. You cannot peer two VPCs with overlapping CIDRs. Plan your IP address space before creating anything — it is much easier to plan on paper than to migrate a live VPC.

AWS Account / VPC	CIDR Block	Subnets	Rationale
Production VPC	10.0.0.0/16	10.0.1-3.0/24 public 10.0.11-13.0/24 app 10.0.21-23.0/24 data	Most IPs needed; largest CIDR; 9 subnets
Staging VPC	10.1.0.0/16	Same structure as prod	Non-overlapping with prod; enables future VPCs
Dev VPC	10.2.0.0/16	10.2.1.0/24 public 10.2.11.0/24 app 10.2.21.0/24 data	Single AZ in dev (cost savings); still non-overlapping
Security VPC	10.3.0.0/16	Minimal subnets (GuardDuty, CloudTrail are added later)	Reserved for security tools; currently minimal usage; room for future
Tools VPC	10.4.0.0/16	10.4.1-3.0/24 public 10.4.11-13.0/24 app	CodePipeline, CodeBuild, CDK bootstrap; could be shared
DR VPC (us-west-2)	10.5.0.0/16	10.5.1-3.0/24 public 10.5.11-13.0/24 app 10.5.21-23.0/24 data	DR region; separate CIDR block; could be shared with prod

■ Rule: Never use 10.0.0.0/8 flat or 172.16.0.0/12 unless you control all IP allocations company-wide

Many companies use 10.0.0.0/16 for every VPC, then discover they cannot connect them together. Assign each VPC a unique /16 from the 10.0.0.0/8 space (10.0, 10.1, 10.2...) and document the allocation in a spreadsheet. This takes 30 minutes upfront and saves days of re-architecture later.

03-21 Elastic Network Interfaces and ENI Limits

Every ECS Fargate task gets its own Elastic Network Interface (ENI) — a virtual network card with a private IP address, security groups, and its own flow logs. Understanding ENI limits is important because running out of ENIs stops ECS from starting new tasks.

ENI Concept	Detail	ShopFlow Implication
ECS Fargate task networking	Each Fargate task gets one ENI in the subnet with a private IP	200 ECS tasks across 3 subnets = 67 ENIs per subnet; /24 has 255 usable IPs
ENI limit per VPC	Soft limit of 5,000 ENIs per VPC (can be increased)	ShopFlow at scale: ~500 tasks + NAT GWs + RDS + ElastiCache = ~1,000 ENIs
ENI and security groups	Each ENI can have up to 5 security groups attached	ECS tasks need: ecs-sg (1 SG). Aurora: aurora-sg (1 SG). Leaky bucket for other services
Warm pool and ENIs	If ECS Fargate warm pool pre-provisioned, ENIs are pre-allocated	Pre-allocating 50 warm tasks = 50 ENIs held even when tasks are not running

03-22 Practical VPC Checklist Before Going Live

Run this checklist before deploying any production workload. Each item takes 2-5 minutes to verify in the console and prevents a category of incidents.

Check	How to Verify in Console	Pass Criteria
VPC Flow Logs enabled	VPC → Your VPCs → shopflow-vpc → Flow logs	At least one flow log sending to CloudWatch Logs
No resources in default VPC	VPC → Your VPCs → select default VPC → check Resources	Zero EC2 instances, ECS tasks, or databases in default VPC
All data subnets have no internet route	VPC → Route Tables → data subnet route tables	Routes 0.0.0.0/0 → local only; no 0.0.0.0/0 entry
Security groups use SG references	VPC → Security Groups → each SG → Inbound rules tab	Source column shows sg-xxxxx (not 0.0.0.0/0 except for sg-xxxxx)
NAT Gateways in each AZ	VPC → NAT Gateways → filter by VPC	3 NAT Gateways, one each in us-east-1a, us-east-1b, us-east-1c
Aurora not publicly accessible	RDS → Databases → shopflow-aurora → Connections	Publicly accessible: No; subnet group shows data subnets
S3 and DynamoDB VPC endpoints	VPC → Endpoints → filter by VPC	com.amazonaws.us-east-1.s3 (Gateway) and com.amazonaws.us-east-1.dynamodb (Interface)
DNS resolution and hostnames enabled	VPC → Your VPCs → shopflow-vpc → Details	DNS hostnames: Enabled; DNS resolution: Enabled (not disabled)

03-23

AWS Network Firewall — Deep Packet Inspection

AWS Network Firewall provides stateful deep packet inspection (DPI) for your VPC. Unlike Security Groups (which filter by IP and port), Network Firewall can inspect the actual content of packets — blocking malware domains, filtering HTTP/HTTPS traffic, and detecting intrusion patterns. ShopFlow uses it to filter outbound traffic from ECS tasks, preventing data exfiltration even if a container is compromised.

■ Network Firewall vs Security Groups

Security Groups say: "Allow TCP port 443 from this IP." Network Firewall can say: "Allow HTTPS to api.stripe.com but block HTTPS to known-malware-domain.com" — it inspects the SNI (domain name) inside the TLS handshake, not just the IP and port. This stops a compromised container from exfiltrating customer data to an attacker's server even if it uses HTTPS on port 443.

Example: ECS task sends HTTPS to evil-exfiltration.com:443. Security Group allows it (port 443 outbound: allowed). Network Firewall intercepts, checks the domain against blacklist, blocks the connection. Customer data never leaves the VPC.

Rule Type	What It Filters	ShopFlow Use Case
5-tuple stateless	IP, port, protocol, direction	Block all outbound traffic except to known AWS service IPs and a few others
Stateful domain list	Domain names (via SNI inspection)	Allow: api.stripe.com, api.sendgrid.com, registry.npmjs.org. Block: evil-exfiltration.com
Stateful Suricata rules	Application-layer patterns (HTTP method, user agent, etc.)	Block HTTP POST to non-approved domains; detect SQL injection
Managed threat intelligence	AWS-maintained lists of known malware/botnets	Auto-updated blacklist; zero configuration to get known-bad domains

03-24

Load Balancer Deep Dive — ALB for ShopFlow

The Application Load Balancer (ALB) is the front door to ShopFlow. It accepts all customer HTTPS traffic and routes it to the correct ECS microservice based on the request path. Understanding ALB deeply is crucial — it is the single point where all optimization, security, and routing decisions happen for incoming traffic.

ALB Listener Rules — HTTPS :443

■ Listener rules — evaluated top to bottom; first match wins

- Priority 1 IF path is /api/orders/* → Forward to: shopflow-order-svc-tg (weight 100)
- Priority 2 IF path is /api/products/* → Forward to: shopflow-product-svc-tg (weight 100)
- Priority 3 IF path is /api/users/* → Forward to: shopflow-user-svc-tg (weight 100)
- Priority 4 IF path is /api/search/* → Forward to: shopflow-search-svc-tg (weight 100)
- Priority 5 IF path is /api/payments/* → Forward to: shopflow-payment-svc-tg (weight 100)
- Priority 6 IF header X-Canary = true → Forward to: shopflow-order-svc-v2-tg (canary)

■ Default action (no rule matched): Forward to shopflow-frontend-tg (React SPA)

Add rule

■ ALB listener rules support weighted routing. During a new deployment, set order-svc-v2-tg weight to 10% and order-svc-tg weight to 90%. Watch error rates in CloudWatch. If v2 is stable, gradually shift to 100% v2. This is blue-green deployment with zero downtime.

03-BP

Best Practices and Anti-Patterns

- ✓ Always design a 3-tier VPC from the start: public (ALB/NAT), private app (ECS), private data (Aurora/Redis). Even for small projects — retrofitting tier isolation later is painful and risky.
- ✓ Put one NAT Gateway per Availability Zone. One shared NAT Gateway is cheaper but creates a single point of failure — if that AZ goes down, all ECS tasks lose outbound internet access.
- ✓ Use Security Groups, not IP ranges, as the source in Security Group rules. "Allow from shopflow-ecs-sg" is more secure and maintainable than "Allow from 10.0.11.0/24" — the IP range could change, but the SG reference follows the resource.
- ✓ Enable VPC Flow Logs from day one. The cost is minimal (CloudWatch Logs ingestion) and the debugging value is immense. You cannot debug what you cannot see.
- ✓ Add S3 and DynamoDB Gateway VPC Endpoints immediately — they are free and save NAT Gateway costs while improving security. No reason to skip.
- ✓ Use /16 CIDR for the VPC and /24 for each subnet. /24 gives 251 usable IPs per subnet — enough for hundreds of ECS tasks per AZ. Larger subnets waste IP space; smaller subnets run out of IPs when you scale.
- Never put Aurora or ElastiCache in public subnets. The data tier should always be in isolated private subnets with no route to the internet. Even if you put a strong password on the database, exposure to the internet is an unacceptable risk.
- Do not use the default VPC for production workloads. The default VPC has a permissive design (all subnets are public by default). Create a dedicated VPC with proper tier isolation for every production workload.
- Avoid CIDR overlap when designing multi-VPC architectures. If Dev VPC uses 10.0.0.0/16 and Prod VPC also uses 10.0.0.0/16, they can never be connected via VPC Peering or Transit Gateway. Plan CIDR ranges across all VPCs upfront.

03-QR

Quick Reference and Interview Q&A

Question	Answer
What is the difference between a public subnet and a private subnet?	A public subnet has a route table entry pointing 0.0.0.0/0 (all internet traffic) at an Internet Gateway. R
Why use 3 NAT Gateways instead of 1?	NAT Gateways are AZ-specific. If you have one NAT Gateway in us-east-1a and that AZ has a failure

Question	Answer
What is the difference between a Security Group and a Network ACL?	Security Groups are stateful (track connections; auto-allow response traffic), applied to individual resources. Network ACLs are stateless, applied to subnets.
How do ECS tasks in private subnets reach the Internet?	Via a NAT Gateway. The ECS task initiates a connection to an external IP. The packet is routed to the NAT Gateway, which then forwards it to the Internet.